Your grade: 100%

Your latest: **100%** • Your highest: **100%** • To pass you need at least 80%. We keep your highest score.

Next item →

Activity: Deploy your AI-powered application

Overview

In this Capstone Project, you will act as the Lead AI Architect for a startup. Your goal is to **build a functional "Intelligent Agent" application** using the Google AI Studio Build section, then transform that prototype into a live, production-ready application. You will audit the agent's performance, forecast the costs of scaling to 1,000 users, deploy the code to the cloud, and establish safety protocols to protect your budget. Along the way, you'll answer questions relating to the project.

This is a comprehensive assignment that requires you to apply the skills you have learned to demonstrate you are ready for real-world AI development.

Scenario

Your team has developed a "Market Research Agent." This agent uses function calling to retrieve stock prices and Google Search to find recent news. It formats the final answer into a clean JSON summary.

Management has approved a Beta launch for **1,000 users**.

- **Usage pattern:** Each user will run 5 research queries per day.
- **Budget:** The Finance team has set a strict limit of **$100 USD per month**.
- **Technical requirement:** The application must be hosted securely, with no API keys exposed in the code.

Your mission is to determine if this launch is financially viable and to deploy the system safely.

Before you begin

Ensure you have the following tabs open in your web browser:

1. **Google AI Studio:** Your development environment.
2. **Google Cloud Console:** Your deployment and management center.
3. **Calculator:** For the cost forecasting step.

Steps

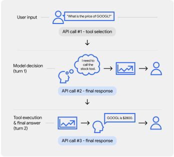
Step 1: The app creation

We will start by creating an application in Google AI Studio.

1. Open **Google AI Studio** and go to the **Build** section.
2. Start a new app and prompt Google AI Studio to create a simple **Stock Research Assistant** with an input box for a stock ticker, a submit button, and a results area. Tell it to add a custom function called `get_stock_price` that takes a required **ticker** string and returns stock data such as the latest price, company name, and price change.
3. Ask it to implement that function on the **server side** so the app calls an external stock API instead of hardcoding stock values.
4. Store the stock API key in **Secrets/environment variables** so it is not exposed in the frontend.
5. Prompt Google AI Studio to enable **Google Search grounding** so the app can find recent articles related to the company or stock ticker.
6. Ask it to combine both tools so the app first gets the stock price through the function call and then uses Google Search to find three recent related articles.
7. Update the output so the app shows: company name and ticker, latest stock price, price movement (if available), and three recent article links with source names.
8. Add a loading state and an error message for invalid tickers or failed API calls.
9. Test the app with sample tickers like **GOOGL** to confirm that function calling retrieves the stock price and Google Search returns related news.

The following diagram illustrates the agentic loop.

The agentic loop (multi-turn cost visualization)


User input
"What is the price of GOOGL?"
API call #1 - tool selection
Model decision (turn 1)
I need to call the stock tool.
API call #2 - final response
Tool execution & final answer (turn 2)
GOOGL is $2800.
API call #3 - final response

Step 2: The token audit

First, you must determine the "cost of intelligence." Agents that use tools consume more tokens than simple chatbots because they send multiple requests (one to ask for the tool, one to process the tool's result).

1. Open your saved **Agent** in Google AI Studio.
    - *Note: If you do not have a saved agent from the previous module, create a new prompt using the **Gemini 3.1 Pro** model and add the **Google Search** tool.*
2. Run a complex test prompt: `Find the current stock price of Google (GOOGL) and summarize the top 3 news headlines about the company from the last 24 hours. Format the output as JSON.`
3. Wait for the result.
4. Select the **Token count** (displayed near the model selection or run button) to view the usage details.
5. Record the **Total Tokens** used for this single interaction (Input + Output).

1. **You are auditing your agent's logs. You notice that for every user query, the agent makes two separate API calls: one to determine which tool to use, and a second to generate the final answer after the tool provides data. How does this "multi-turn" behavior affect the application's production budget? Select the best answer.** 1 / 1 point

○ It decreases costs because the model splits the work into smaller chunks.
○ It has no effect because Google only charges for the final answer.
● It increases costs because the model must process the input tokens and context multiple times to complete the full loop.
○ It increases the cost only if the user rejects the final answer.

✓ Nice work

Correct! Each step in the agent loop counts as a request. Input tokens are sent, and output tokens are generated for *both* the tool decision and the final response.

Step 3: Cost forecasting

You have your baseline data. Now, calculate if the project fits the budget.

1. **The formula:** $(tokens\ per\ query \times queries\ per\ day \times days\ per\ month \times users) \div 1,000,000 \times price\ per\ million$.
2. **The data:**
    - **Tokens per query:** Use the number you found in Step 1 (e.g., 2,000 tokens).
    - **Volume:** 5 queries/day $\times$ 30 days $\times$ 1,000 users = 150,000 queries per month.
    - **Price:** Assume Gemini 3.1 Pro costs roughly **$0.075** per 1 million tokens. Token cost estimates can vary for models. Double check cost estimate posted on Google documentation.
3. **Calculate:**
    - Total monthly tokens = 2,000 $\times$ 150,000 = 300,000,000 tokens.
    - Estimated cost = $(300,000,000 \div 1,000,000) \times \$0.075$

Decision Point: Is the result under $100? If yes, proceed. If 'no', you would need to switch to a cheaper model or shorten your system instructions.

2. **Your cost forecast predicts a monthly bill of $500, which is well over the $100 budget. You examine your prompt and see you are sending 50 pages of documentation as "context" with every single request. Which strategy would most effectively reduce costs while maintaining the application's ability to answer questions? Select the best answer.** 1 / 1 point

● Implement a retrieval system (RAG) to only send the specific paragraph needed for the user's question, rather than the whole document.
○ Remove the documentation entirely and rely on the model's training data.
○ Ask the users to type shorter questions.
○ Switch to the Gemini Pro model, which has a larger context window.

✓ Nice work

Correct! Sending only relevant text significantly reduces the "Input Token" count per request, lowering costs without losing accuracy.

Step 4: Secure production deployment

You have validated the functionality and the budget. Now, deploy the agent to the web.

1. Make sure you are in the Build section of your Google AI Studio.
2. Select the **Publish** button.
3. **Project Settings:** Select your **Google Cloud Project ID** and a **Region** (e.g., `us-central1`).
4. **API Key Security:** Notice that Google AI Studio asks how to handle the API key. It will automatically configure it as an **Environment Variable**.
5. Select **"Publish App" under the Publish section on the top right section.**
6. Wait for the build process to finish. When complete, copy the **Service URL** provided. This is your live agent.

3. **You are reviewing the code for a junior developer's AI application. You notice they have pasted their string of API key characters directly into the `app.py` file inside quotes. Why must you ask them to change this before the application is deployed? Select the best answer.** 1 / 1 point

○ Google Cloud Run cannot read text files, so the application will crash.
● Hardcoding credentials allows anyone who sees the code to steal the key and use the quota, potentially incurring high costs.
○ It causes the model to respond slower because it has to read the key every time.
○ The API key will expire after 24 hours if it is written in the code.

✓ Nice work

Correct! This is a critical security vulnerability. Keys should always be stored in environment variables or secret managers, never in the code itself.

Step 5: Financial safety switches

In a production environment, code can sometimes loop unexpectedly, or a post might go viral, bringing in 10,000 users instead of 1,000. You must set a safety net.

1. Open the **Google Cloud Console**.
2. Navigate to **Billing** > **Budgets & alerts**.
3. Create the budget:
    a. Select **Create budget.**
    b. Give your budget a name.
    c. Set the Scope for your budget. Select the Project where your application is set to.
    d. Set the Amount. You can specify a fixed amount or base it on the previous period's spending.
4. Set up alert rules:
    a. Under Actions, set the Alert threshold rules. By default, alerts are set for 50%, 90%, and 100% of your budget amount based on actual spend.
    b. You can customize these rules by changing the percentage and choosing to trigger on Actual or Forecasted.
5. Configure notifications and save:
    a. Under Manage notifications, choose who receives the email alerts.
    b. Select **Finish** to save your budget.

4. **You receive a billing alert at 3:00 AM stating that 100% of your budget has been used. You check the logs and see a single user has sent 50,000 requests in one hour. What is the most likely cause of this anomaly? Select the best answer.** 1 / 1 point

○ The Gemini model got stuck in a "thinking" loop.
○ The user is a very fast reader and is researching a complex topic.
● The user is likely a bot or a script that is programmatically calling your API endpoint.
○ The Google Cloud billing system made a calculation error.

✓ Nice work

Correct! High-volume, rapid-fire requests usually indicate automated traffic. You should implement rate limiting to block this behavior.

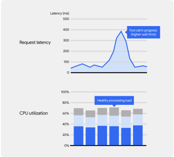
Step 6: Monitoring and optimization

The app is live. Now you must watch its health.

1. In the Publish Section, select **Advanced Settings.**
2. Get redirected to the app's **Cloud Run** section.
3. Select the **Metrics** tab.
4. Locate the **Request Count** and **Request Latency** charts.

The following dashboard shows how latency increases during tool use while CPU remains stable.

Production health dashboard



5. **Users are complaining that the agent takes 10 seconds to answer simple questions like "What is your name?". You check the logs and see that for every question, the agent is searching the web before answering. How should you adjust the agent's configuration to fix this latency issue? Select the best answer.** 1 / 1 point

○ Remove the Google Search tool entirely from the application.
○ Increase the model's temperature to make it answer faster.
○ Upgrade the server to a CPU with more cores.
● Refine the system instructions to explicitly tell the model: "Only use Google Search for current events. Do not use tools for general chat."

✓ Nice work

Correct! Proper prompt engineering can teach the model when to use a tool and when to skip it, optimizing speed for simple queries.

Exemplar: Lead architect's final report

Compare your findings with this sample "Gold Standard" report for the Market Research Agent launch.

1. Token audit results

- **Sample input:** "Research GOOGL and summarize news."
- **Turn 1 (tool selection):** 450 tokens
- **Turn 2 (tool output and final generation):** 1,550 tokens
- **Total audit: 2,000 tokens per full cycle**

2. Financial forecast for one month

- **Usage:** 1,000 users × 5 queries per day × 30 days = **150,000 queries**
- **Token volume:** 150,000 queries × 2,000 tokens = **300 million tokens**
- **Cost analysis:** 300 million tokens × **$0.075 per million** = **$22.50**
- **Viability: Approved.** The projected cost ($22.50) is well under the $100 budget. This leaves a significant buffer for usage spikes or higher-than-expected output length.

3. Deployment and limits

- **Platform:** Google Cloud Run
- **Security:** API Keys are stored in **environment variables** to prevent exposure in the source code.
- **Safety limit:** A budget alert is set at **$100**. If the application hits **$50 (50%)** by the 5th day of the month, the Architect knows a bot or an error loop is occurring and can intervene before the limit is reached.

4. Optimization observations

- **Latency spike:** A **2.5-second spike** occurred during news summarization.
- **Diagnosis:** The delay is caused by the model waiting for the **Google Search tool** to return web data.
- **Recommendation:** To improve the user experience, add a **"Thinking" indicator** in the user interface. This tells users the agent is actively searching the web and prevents them from refreshing the page.

Success checklist

- **Create:** Developed a prototype app.
- **Audit:** Calculated the token usage for a multi-step agent task.
- **Forecast:** Produced a math-based cost estimate for a 1,000-user rollout.
- **Deploy:** Launched an application to a Cloud Run URL.
- **Safety:** Established a Google Cloud budget alert.
- **Review:** Analyzed latency metrics to identify performance bottlenecks.

Reflection

- How does the "Cost per Token" model change how you design features? Do you feel pressure to make the AI more concise?
- If you had to lower costs by 50%, would you choose a cheaper model (potentially lower quality) or remove features (lower utility)?

Next steps

You have moved from writing your first prompt to architecting, coding, and deploying a production-grade AI agent. You are now ready to integrate these skills into your professional portfolio. Continue experimenting with the Gemini API as new models and features are released!